



ACCRA TECHNICAL UNIVERSITY, GHANA
FACULTY OF APPLIED SCIENCES
DEPARTMENT OF COMPUTER SCIENCE
BTECH COMPUTER SCIENCE — YEAR II
COURSEWORK ASSIGNMENT 2

BCP 210: DATA STRUCTURES AND ALGORITHMS I
ACADEMIC YEAR: 2025/2026

STUDENT NAME: _____ INDEX NUMBER: _____

INSTRUCTIONS TO STUDENTS:

1. Answer **ALL** questions. The assignment is marked out of **100 marks**.
 2. Implement all algorithms in Python (version 3.8 or above).
 3. You may use only Python's standard built-in types (`list`, `dict`, `set`, etc.). **No external libraries** (e.g., `numpy`, `pandas`) are permitted unless explicitly stated.
 4. **GitHub Submission Steps:**
 - (a) Go to the starter repository: <https://github.com/atu-dsa-2025/dsa-assignment-2-template>
 - (b) Click the green “Use this template” button → select “Create a new repository”.
 - (c) Set the repository visibility to **Private** and name it `dsa-assignment-2-INDEXNUMBER`.
 - (d) Commit and push your Python source files (`.py`) and a written report (`.pdf`) to your repository.
 - (e) **Transfer ownership:** When finished, go to **Settings** → **Danger Zone** → **Transfer ownership** and transfer the repository to the class organisation: `atu-dsa-2025`.
 5. **Submission Verification:** Your submission date is verified strictly by the GitHub server push timestamp. Local commit timestamps will not be accepted as proof of on-time submission.
 6. **Late Policy:** Submissions pushed after the deadline will incur a penalty of 10% per 24-hour delay.
-

1 Part A: Algorithmic Complexity Analysis (20 Marks)

Topic: Analysing and Comparing Algorithm Efficiency

Background

A university exam system uses three candidate algorithms to search for a student's record in a database of N entries.

```

1 # Algorithm X
2 def algorithm_x(records, target):
3     for i in range(len(records)):
4         for j in range(i, len(records)):
5             if records[i] + records[j] == target:
6                 return (i, j)
7     return None
8
9 # Algorithm Y
10 def algorithm_y(records, target):
11     seen = {}
12     for i, val in enumerate(records):
13         complement = target - val
14         if complement in seen:
15             return (seen[complement], i)
16         seen[val] = i
17     return None
18
19 # Algorithm Z
20 def algorithm_z(records):
21     n = len(records)
22     for i in range(1, n):
23         key = records[i]
24         j = i - 1
25         while j >= 0 and records[j] > key:
26             records[j + 1] = records[j]
27             j -= 1
28         records[j + 1] = key
29     return records

```

Questions

- A1. (4 Marks)** Determine the **worst-case time complexity** of `algorithm_x` and justify your answer by explaining which part of the code drives the cost.
- A2. (4 Marks)** Determine the **worst-case time complexity** of `algorithm_y` and explain what data structure makes it faster than `algorithm_x`. What is the space trade-off?
- A3. (4 Marks)** `algorithm_z` sorts an array and is well-known for performing well on nearly-sorted inputs. Name this algorithm. State its **best-case** and **worst-case** time complexities and explain what input arrangement causes each case.
- A4. (4 Marks)** The exam system processes $N = 1,000,000$ student records. Complete the table below with the **approximate number of operations** for each complexity class at this scale, and rank the algorithms from fastest to slowest.

Complexity Class	Approx. Operations at $N = 10^6$	Rank (1 = fastest)
$O(1)$	_____	_____
$O(\log N)$	_____	_____
$O(N)$	_____	_____
$O(N \log N)$	_____	_____
$O(N^2)$	_____	_____

- A5. (4 Marks)** A recursive function computes $f(n) = f(n - 1) + f(n - 2)$ with base cases $f(0) = 0$ and $f(1) = 1$, and no result is cached. Explain why its time complexity is $O(2^N)$ rather than $O(N)$. How would you reduce it to $O(N)$ time and $O(1)$ space?

2 Part B: Arrays and the Two-Pointer Technique (20 Marks)

Topic: ATU Library Book Management System

Background

The ATU library maintains its catalogue as a **sorted** integer array where each integer encodes a unique `Book_ID`. The following sorted array is provided for all questions in this section:

```
1 catalogue = [-8, -3, 0, 1, 4, 6, 9, 12, 15, 21]
```

Questions

- B1. (5 Marks)** Implement a function `binary_search(arr, target)` that returns the index of `target` in the sorted array, or `-1` if it is not found. Your implementation must run in $O(\log N)$ time. State its best-case and worst-case complexities and describe what causes each.
- B2. (6 Marks)** Implement a function `find_pair_with_sum(arr, target)` that, using the **two-pointer technique**, finds two distinct Book IDs in the sorted catalogue whose values sum to a given `target`. The function should return the pair as a tuple, or `None` if no such pair exists. It must run in $O(N)$ time and $O(1)$ additional space. Demonstrate your function by finding a pair that sums to **13**.
- B3. (5 Marks)** Implement a function `rotate_array(arr, k)` that rotates the array to the **right** by k positions **in-place** (without creating a copy of the array). For example, rotating `[1, 2, 3, 4, 5]` by $k = 2$ produces `[4, 5, 1, 2, 3]`. Your solution must run in $O(N)$ time and $O(1)$ space. (*Hint: consider a triple-reversal approach.*)
- B4. (4 Marks)** Explain what **amortised** $O(1)$ means in the context of appending elements to a Python list. Why does a single append occasionally trigger an $O(N)$ operation, and why is the long-run average still considered constant time?

3 Part C: Linked Lists, Stacks, and Queues (25 Marks)

Topic: ATU Campus Shuttle Booking System

Background

The ATU campus shuttle booking system manages passenger reservations using a **Doubly Linked List**. Each booking node holds:

- `booking_id` (Integer)
- `student_name` (String)
- `destination` (String)
- `next` and `prev` pointers

Additionally, the shuttle dispatch office uses a **Stack** to track the history of route changes and a **Queue** to manage the boarding order at each stop.

Questions

Section C1: Doubly Linked List (15 Marks)

- C1. (3 Marks)** Implement the `Booking` node class and the `ShuttleList` class with an `__init__` method that maintains both a `head` and a `tail` pointer.
- C2. (4 Marks)** Implement `add_booking(booking_id, student_name, destination)` which appends a new booking to the **end** of the doubly linked list. Correctly set both `next` and `prev` pointers. State the time complexity of this operation when a `tail` pointer is maintained, and explain how it would change without one.
- C3. (4 Marks)** Implement `cancel_booking(booking_id)` which removes a booking node from **any position** in the list (head, tail, or middle) by adjusting both the `next` and `prev` pointers of the surrounding nodes. Your implementation must handle all three cases correctly.
- C4. (4 Marks)** Implement `find_and_swap(id1, id2)` which locates the two booking nodes with the given IDs and swaps their data (`booking_id`, `student_name`, `destination`) **without** relinking any pointers. State the time complexity of your solution and explain why swapping data is preferred over relinking pointers in this case.

Section C2: Stack and Queue Application (10 Marks)

- C5. (5 Marks)** The dispatch office records every route change as a string (e.g., "Route A -> Route B") by pushing it onto a `Stack`. When the office wants to **undo** the last change, it pops the top. Implement a `RouteHistory` class backed by a Python list (used as a stack) that supports:
- `push(change)`: Record a new route change.
 - `pop_undo()`: Undo and return the most recent change. Return `None` if nothing to undo.
 - `peek()`: View the most recent change without removing it.

State the time complexity of each operation.

C6. (5 Marks) At each shuttle stop, students board in a strict First-Come-First-Served order. Implement a `BoardingQueue` class using Python's `collections.deque` that supports:

- `join(student_name)`: A student joins the back of the queue.
- `board()`: The next student boards (leaves the front). Return `None` if the queue is empty.
- `peek_next()`: View the next student to board without removing them.
- `size()`: Return the number of students currently in the queue.

Explain why `collections.deque` is preferred over a plain Python `list` for implementing a queue.

4 Part D: Hash Tables and Binary Search Trees (25 Marks)

Topic: ATU Student Grade Management Portal

Background

The grade management portal stores course results in a **Hash Table** for fast retrieval. An internal audit requires a BST to be built from the data so that grade reports can be printed in sorted order without re-sorting the entire dataset.

Section D1: Hash Tables (12 Marks)

- D1. (3 Marks)** Explain the concept of a **hash collision**. Describe the two standard resolution strategies (**Chaining** and **Open Addressing** / Linear Probing) and state the worst-case time complexity for a lookup under each strategy.
- D2. (5 Marks)** Implement a function `grade_frequency_report(results)` that takes a list of tuples of the form `(student_id, grade_letter)` and returns a dictionary mapping each grade letter ('A', 'B', 'C', 'D', 'F') to the number of students who received it. Use the following sample data to demonstrate your function:

```
1 results = [  
2     (1001, 'A'), (1002, 'B'), (1003, 'A'), (1004, 'C'),  
3     (1005, 'B'), (1006, 'A'), (1007, 'F'), (1008, 'B'),  
4     (1009, 'C'), (1010, 'A'),  
5 ]  
6
```

Expected output: {'A': 4, 'B': 3, 'C': 2, 'D': 0, 'F': 1}

- D3. (4 Marks)** Implement a function `find_students_with_grade(results, grade)` that returns a **sorted list** of student IDs for students who received the specified grade. Your solution must run in $O(N \log N)$ time. State why the time complexity is not $O(N)$ despite using a hash table for grouping.

Section D2: Binary Search Trees (13 Marks)

The grade portal builds a BST where each node stores a unique `(student_id, grade_score)` pair (an integer 0–100) keyed on `grade_score`.

```
1 class BSTNode:  
2     def __init__(self, student_id, grade_score):  
3         self.student_id = student_id  
4         self.grade_score = grade_score  
5         self.left = None  
6         self.right = None
```

- D4. (4 Marks)** Implement a recursive function `insert(root, student_id, grade_score)` that inserts a new node into the BST following standard BST ordering on `grade_score`. Demonstrate by inserting the following records in order: (1001, 72), (1002, 55), (1003, 88), (1004, 60), (1005, 95), (1006, 48). Draw the resulting tree structure.
- D5. (4 Marks)** Implement `inorder_traversal(root)` as a generator function (using `yield`) that visits nodes in ascending order of `grade_score`. Show the output of calling this function on the BST you built in D4. Explain why in-order traversal produces a sorted sequence in a BST.

- D6. (5 Marks)** Implement `search(root, grade_score)` that returns the `student_id` of the node matching the given score, or `None` if not found. Then implement `find_range(root, low, high)` that returns a **list** of all `student_id` values whose `grade_score` falls within the inclusive range $[low, high]$, in ascending order. State the time complexity of each function in terms of N (number of nodes) and H (tree height), and explain when $H = O(\log N)$ versus $H = O(N)$.

5 Bonus Challenge (10 Marks)

Integrating Data Structures: Exam Invigilation Scheduler

Background

The ATU examinations office must assign invigilators to exam rooms. They have a list of exam sessions, each represented as a tuple (`room_id`, `start_time`, `end_time`) where all times are integers. Two sessions **conflict** if they share the same `room_id` and their time intervals overlap (i.e., one session starts before the other ends).

Problem Statement

- E1. (4 Marks)** Implement `build_conflict_map(sessions)` that returns a dictionary mapping each `room_id` to a list of **all** pairs of conflicting session indices within that room. Two sessions at indices i and j (where $i < j$) conflict if `start[j] < end[i]`. Use the sample data below:

```
1 sessions = [  
2     ('R1', 8, 10),    # index 0  
3     ('R1', 9, 11),    # index 1 -- conflicts with index 0  
4     ('R1', 11, 13),   # index 2 -- no conflict with index 1  
5     ('R2', 8, 12),    # index 3  
6     ('R2', 10, 14),   # index 4 -- conflicts with index 3  
7 ]  
8
```

- E2. (3 Marks)** Using a **Stack**, implement `detect_first_conflict(room_sessions)` that processes sessions for a **single room** in order of start time and returns the **first conflicting pair** of session indices found, or `None` if no conflict exists.
- E3. (3 Marks)** Analyse the overall time and space complexity of your full solution to E1. Discuss whether a BST keyed on `start_time` or a sorted array with binary search would improve the performance of conflict detection compared to the naive nested-loop approach.

Mark Allocation Summary

Table 1: Coursework Assignment 2 — Mark Distribution

Part	Topic (Weeks Covered)	Marks
Part A	Algorithmic Complexity Analysis (Week 1)	20
Part B	Arrays and Two-Pointer Technique (Week 2)	20
Part C	Linked Lists, Stacks, and Queues (Weeks 3 & 4)	25
Part D	Hash Tables and Binary Search Trees (Weeks 5 & 6)	25
Part E	Bonus: Exam Invigilation Scheduler (Weeks 1–6 integrated)	10
Total		100 (+10 bonus)